ANALIZADOR DE FORMULARIOS

MANUAL TECNICO

Presentación

El siguiente manual se ha desarrollado con la finalidad de dar a conocer la información necesaria para realizar mantenimiento, instalación y exploración del software, el cual consta de diferentes funcionalidades. El manual ofrece la información necesaria de ¿Cómo está realizado el software? Para que la persona (Desarrollador/Ingeniero) que quiera editar el software lo haga de una manera apropiada, dando a conocer la estructura del desarrollo del aplicativo. La siguiente guía se encuentra dividida en las herramientas que se usaron para la creación del software con una breve explicación paso a paso.

Introducción

El manual se realiza con el fin de detallar el software para el Analizador de Formularios en términos técnicos para que la persona que vaya a administrar, editar o configurar el aplicativo lo haga de una manera apropiada. El documento se encuentra dividido en las siguientes secciones:

- ASPECTOS TEORICOS: Se dará a conocer conceptos, definiciones y explicaciones de los componentes del aplicativo desde un punto de vista teórico para mayor entendimiento por parte del lector sobre el funcionamiento del sistema de información y herramientas.
- DIAGRAMAS DE MODELAMIENTO: Se compone por diagramas e ilustraciones alusivas al funcionamiento del aplicativo (Los diagramas se tienen en archivos diferentes al actual, buscar las referencias).

ASPECTOS TECNICOS

El aplicativo tiene como finalidad interpretar formularios a su respectivo renderizado. Se recomienda que el siguiente manual sea manipulado únicamente por la persona que quiera administrar, editar o configurar el software para velar por la seguridad de los daos que se almacenan.

1. HERRAMINETAS UTILIZADAS: En esta sección se procede a explicar las herramientas informáticas empleadas para el desarrollo del aplicativo:
    a. Android Studio: Android Studio es el entorno de desarrollo integrado (IDE) oficial creado por Google para diseñar, codificar, probar y depurar aplicaciones para la plataforma Android. Basado en IntelliJ IDEA,

proporciona herramientas como un emulador rápido, sistema de compilación Gradle y soporte para Kotlin, Java y C++. Para la descarga e Instalación del IDE puede ir a su página oficial y seguir la guía de instalación https://developer.android.com/studio/install?hl=es-419

b. JFlex: Es un generador de analizadores léxicos (escáneres) de código abierto, rápido y eficiente, diseñado específicamente para Java. Toma un archivo de especificación con expresiones regulares y genera código Java que clasifica el texto de entrada en tokens (palabras clave, operadores, etc.), siendo fundamental en la creación de compiladores e intérpretes. Para la descarga e Instalación de la herramienta puede ir a su página oficial y descargar el archivo comprimido, para posteriormente extraer el archivo .jar https://www.jflex.de/download.html

c. Java CUP: Es una herramienta diseñada para la creación de analizadores sintácticos (parsers) en el lenguaje de programación Java. Utiliza una gramática LALR(1) para generar código Java que puede reconocer estructuras de lenguaje, siendo frecuentemente usado junto con JFlex para el análisis léxico. Para la Descarga e Instalación de la herramienta puede ir a su página oficial y descargar el archivo comprimido, para posteriormente extraer los archivos .jar https://www2.cs.tum.edu/projects/cup/install.php

2. ESTRUCTURA DEL PROYECTO: En esta sección se procederá a explicar las clases/archivos que tienen una relevancia significativa en el funcionamiento del software:

   a. FormAnalyzer.kt: Esta clase es la encargada de interactuar con las clases LexerForm y ParserForm (generadas por JFlex y CUP respectivamente), para obtener los resultados del análisis al código .form ingresado.

   b. LexerForm.java: Clase generada por JFlex, la cual contiene el Autómata Finito Determinista para el análisis léxico del código .form de entrada. Esta clase se genera en base al archivo de configuración especificado para el análisis léxico.

   c. ParserForm.java: Clase generada por CUP, la cual contiene el código necesario para el análisis sintáctico del codigo .form de entrada. Esta clase se genera en base al archivo de configuración especificado para el análisis sintáctico.

   d. LexerForm.flex: Archivo de configuración para que JFlex genere una Clase Java que hará el análisis léxico en base al archivo de configuración.

   e. ParserForm.cup: Archivo de configuración para que CUP genere una Clase Java que hará el análisis sintáctico en base al archivo de configuración.

f. FormElement.kt: Clase que será la base para definir cada tipo de objeto/elemento dentro del formulario ya renderizado.
g. BorderAttributes.kt, StyleAttributes.kt: Clases que heredan de FormElement implementando atributos específicos para representar específicamente a los atributos que tendrá cada tipo de elemento renderizado dentro del formulario.
h. TableElement.kt, TextElement.kt, SectionElement.kt, DropQuestionElement.kt, MultipleQuestionElement.kt, OpenQuestionElement.kt, SelectQuestionElement.kt : Clases que heredan de FormElement implementando atributos especifico que van a representar a cada elemento grafico dentro del renderizado final del formulario.
i. BorderType.kt, ColorType.kt, FontFamily.kt, OperatorType.kt, OrientationType.kt, PokemonQueryType.kt, ValueType.kt, VariableType.kt: Son enums que contienen constantes, tales como, los tipos de borde que tendrán algunos elementos, el nombre de colores y fuentes fijos que se van a manejar en el diseño, el tipo de operaciones que soportará el lenguaje, las orientaciones que se tendrá en el flujo de diseño del formulario, los tipos de valores y variables que se tendrán en el lenguaje .form.
j. ErrorReport.kt: Clase que modela los datos del reporte que se van a representar como parte del resultado del análisis al código .form.
k. Interpreter.kt: Clase core de la aplicación, se encarga de recibir el ast del ParserForm, para posteriormente ejecutar y evaluar cada expresión o declaración dependiendo el tipo que sea, esta clase es la encargada de tener la lógica de negocio, se encarga de que se cumpla con los requerimientos de análisis semántico, al concluir con toda la interpretación del ast devuelve una lista de FormElement el cual servirá para renderizar los elementos del formulario.
l. Enviroment.kt: Clase encargada de manjar los ámbitos dentro del lenguaje .form, es la clase encargada de lleva el control de las variables y funciones utilizadas dentro de cada ámbito dentro del código.
m. SymbolValue.kt: Clase que representa a un Identificador dentro de la Tabla de Símbolos que va a manejar el Enviroment
n. FormRendered.kt: Es la clase encargada de Renderizar cada FormElement que recibe del Interpreter y mostrarlo en la interfaz de la aplicación, prácticamente esta clase maneja todo el diseño del formulario resultante del análisis hecho.
o. AST.kt: Es la clase base que representara a los diferentes nodos resultantes del ParserForm

p. Program.kt: Clase que envolverá los nodos del ParserForm en una lista para su posterior interpretación en el Interpreter.
q. Expression.kt: Clase que hereda de AST el cual va a representar a una expresión dentro del lenguaje .form. De esta clase heredan más clases que representaran de forma específica a cada tipo de expresión que se va a tener dentro del lenguaje.
r. Statement.kt: Clase que hereda de AST el cual va a representar a una declaración dentro del lenguaje .form. De esta clase heredan más clases que representaran de forma específica a cada tipo de declaración que se va a tener dentro del lenguaje.
s. DropQValue.kt, MultipleQValue.kt, OpenQValue.kt, SelectQValue.kt, SpecialQValue.kt: Estas clases representan los valores Especiales de tipo pregunta, son como el intermediario entre la entrada en código y el resultado final del formulario.
t. StyleValue.kt: Esta sealed class representa a un estilo como si fuera un valor final más dentro del lenguaje, de esta sealed class heredan los estilos especificos tales como el color, la fuente, el tamaño de texto y el estilo de borde.
u. OptionsSource.kt: Esta sealed class es un envoltorio para saber el tipo de opciones que se va a tener en el DropQuestion, ya que se pueden tener opciones del tipo literal (valores planos como number y string) y las opciones que vendrán de la PokeAPI
v. TableActivity.kt: Clase que genera/construye las tablas de reportes en base a las listas de objetos de reporte que obtiene del análisis realizado al código .form.
w. ContentActivity.kt: Sera la vista principal de la aplicación, ya que aquí es en donde se tendrá al editor de código y a los elementos renderizados del formulario.
x. MainActivity.kt: Clase principal que controla el flujo de navegación de la aplicación.

3. GRAMATICA PARA ANALIZADOR LEXICO: En esta sección se procede a explicar parte esencial del archivo de configuración .flex para generar el análisis léxico para el lenguaje .form, es decir, se va a explicar las expresiones regulares utilizadas para el análisis:
   a. Constantes:

```
// REGULAR EXPRESSIONS
LineTerminator = \r|\n|\r\n
WhiteSpace = [ \t\f]
InputCharacter = [^\r\n]
Dot = \.
WholeNumber = 0|[1-9][0-9]*
DecimalNumber = {WholeNumber}{Dot}[0-9]+
```

b. Palabras Reservadas:

```
"number" { return symbol(symForm.NUMBER); }
"string" { return symbol(symForm.STRING); }
"special" { return symbol(symForm.SPECIAL); }

"OPEN_QUESTION" { return symbol(symForm.OPEN_QUESTION); }
"SECTION" { return symbol(symForm.SECTION); }
"TABLE" { return symbol(symForm.TABLE); }
"TEXT" { return symbol(symForm.TEXT); }
"DROP_QUESTION" { return symbol(symForm.DROP_QUESTION); }
"SELECT_QUESTION" { return symbol(symForm.SELECT_QUESTION); }
"MULTIPLE_QUESTION" { return symbol(symForm.MULTIPLE_QUESTION); }

"RED" { return symbol(symForm.RED); }
"BLUE" { return symbol(symForm.BLUE); }
"GREEN" { return symbol(symForm.GREEN); }
"PURPLE" { return symbol(symForm.PURPLE); }
"SKY" { return symbol(symForm.SKY); }
"YELLOW" { return symbol(symForm.YELLOW); }
"BLACK" { return symbol(symForm.BLACK); }
"WHITE" { return symbol(symForm.WHITE); }

"VERTICAL" { return symbol(symForm.VERTICAL); }
"HORIZONTAL" { return symbol(symForm.HORIZONTAL); }

"MONO" { return symbol(symForm.MONO); }
"SANS_SERIF" { return symbol(symForm.SANS_SERIF); }
"CURSIVE" { return symbol(symForm.CURSIVE); }

"LINE" { return symbol(symForm.LINE); }
"DOTTED" { return symbol(symForm.DOTTED); }
"DOUBLE" { return symbol(symForm.DOUBLE); }
```

```
"width" { return symbol(symForm.WIDTH); }
"height" { return symbol(symForm.HEIGHT); }
"label" { return symbol(symForm.LABEL); }
"draw" { return symbol(symForm.DRAW); }
"pointX" { return symbol(symForm.POINT_X); }
"pointY" { return symbol(symForm.POINT_Y); }
"orientation" { return symbol(symForm.ORIENTATION); }
"elements" { return symbol(symForm.ELEMENTS); }
"styles" { return symbol(symForm.STYLES); }
"content" { return symbol(symForm.CONTENT); }
"options" { return symbol(symForm.OPTIONS); }
"correct" { return symbol(symForm.CORRECT); }
"who_is_that_pokemon" { return symbol(symForm.WHO_IS_THAT_POKEMON); }

"NUMBER" { return symbol(symForm.NUMBER_POKEMON); }

\""color"\" { return symbol(symForm.COLOR); }
\""background color"\" { return symbol(symForm.BACKGROUND_COLOR); }
\""font family"\" { return symbol(symForm.FONT_FAMILY); }
\""text size"\" { return symbol(symForm.TEXT_SIZE); }
\""border"\" { return symbol(symForm.BORDER); }

"IF" { return symbol(symForm.IF); }
"ELSE" { return symbol(symForm.ELSE); }
"WHILE" { return symbol(symForm.WHILE); }
"DO" { return symbol(symForm.DO); }
"FOR" { return symbol(symForm.FOR); }
"in" { return symbol(symForm.IN); }
```

c. Operadores Relacionales:

```
"==" { return symbol(symForm.IGUALDAD); }
"!!" { return symbol(symForm.DIFERENTE); }
">"  { return symbol(symForm.MAYOR); }
"<"  { return symbol(symForm.MENOR); }
">=" { return symbol(symForm.MAYOR_IGUAL); }
```

d. Operadores Lógicos:

```
"&&" { return symbol(symForm.AND); }
"||" { return symbol(symForm.OR); }
"~"  { return symbol(symForm.NOT); }
```

e. Operadores Matemáticos:

```
"+" { return symbol(symForm.SUMA); }
"-" { return symbol(symForm.RESTA); }
"*" { return symbol(symForm.MULTIPLICACION); }
"/" { return symbol(symForm.DIVISION); }
"^" { return symbol(symForm.POTENCIA); }
"%" { return symbol(symForm.MODULO); }
"(" { return symbol(symForm.PARENTESIS_ABIERTO); }
")" { return symbol(symForm.PARENTESIS_CERRADO); }
```

f. Colores Hexadecimales:

```
"#"[a-fA-F0-9]{6} {
    this.string.setLength(0);
    this.string.append(yytext());
    this.string.deleteCharAt(0);
    return symbol(symForm.HEXADECIMAL, this.string.toString().trim());
}
"#"[a-fA-F0-9]{3} {
    this.string.setLength(0);
    this.string.append(yytext());
    this.string.deleteCharAt(0);
    return symbol(symForm.HEXADECIMAL, this.string.toString().trim());
}
```

g. Expresiones Regulares con Estados:

```
"$" { this.string.setLength(0); yybegin(SIMPLE_COMMENT); }
"/*" { this.string.setLength(0); yybegin(MULTIPLE_COMMENT); }

\" { this.string.setLength(0); yybegin(TEXT); }
```

```
// Comments states
<SIMPLE_COMMENT> {
    {InputCharacter}+ { this.string.append(yytext()); }
    {LineTerminator} { yybegin(YYINITIAL); }
    <<EOF>> { yybegin(YYINITIAL); }
}
<MULTIPLE_COMMENT> {
    "*/" {
        System.out.println("COMENTARIO MULTILINEA: " + this.string.toString().trim());
        yybegin(YYINITIAL);
    }
    {InputCharacter} { this.string.append(yytext()); }
    {LineTerminator} { /* Ignore */ }
}

// Text state
<TEXT> {
    \" {
        System.out.println("TEXTO: " + this.string.toString().trim());
        yybegin(YYINITIAL);
        return symbol(symForm.TEXTO, this.string.toString().trim());
    }
    {InputCharacter} { this.string.append(yytext()); }
    {LineTerminator} { this.string.append("\n"); }
}
```

h. Expresiones Regulares Útiles:

```
{WholeNumber}   { return symbol(symForm.NUMERO_ENTERO, Integer.parseInt(yytext())); }
{DecimalNumber} { return symbol(symForm.NUMERO_DECIMAL, Double.parseDouble(yytext())); }
{ID}            { return symbol(symForm.ID, yytext()); }
{Dot}           { return symbol(symForm.PUNTO); }
```

4. GRAMATICA PARA ANALIZADOR SINTACTICO: En esta sección se procede a presentar parte esencial del archivo de configuración .cup para generar el análisis sintáctico dentro de la aplicación, es decir, se va a presentar las gramáticas utilizadas para el análisis: (Nota: Los símbolos Terminales están en Mayúscula y los símbolos No Terminales están en Minúscula)

```
start with contenido;

contenido ::= contenido instruccion |
            contenido error |
            instruccion;
instruccion ::= variable |
            invocacion |
            seccion |
            tabla |
            pregunta |
            bloque_codigo;

bloque_codigo ::= if |
              while |
              do |
              for;

variable ::= tipo_dato ID asignacion |
         ID IGUAL tipo_valor;

tipo_dato ::= NUMBER |
            STRING |
            SPECIAL ;

asignacion ::= IGUAL tipo_valor |
            /* empty */;
```

```
tipo_valor ::= valor;

valor ::= operacion_matematica |
      pregunta;

invocacion ::= ID PUNTO DRAW PARENTESIS_ABIERTO parametros_draw;

parametros_draw ::= PARENTESIS_CERRADO |
              lista_parametros PARENTESIS_CERRADO;

lista_parametros ::= lista_parametros COMA operacion_matematica |
              lista_parametros COMA error |
              operacion_matematica;

color ::= HEXADECIMAL |
      rgb |
      hsl |
      color_literal;

rgb ::= PARENTESIS_ABIERTO operacion_matematica COMA operacion_matematica
COMA operacion_matematica PARENTESIS_CERRADO;

hsl ::= MENOR operacion_matematica COMA operacion_matematica COMA
operacion_matematica MAYOR;

color_literal ::= RED |
            BLUE |
            GREEN |
            PURPLE |
            SKY |
            YELLOW |
            BLACK |
            WHITE;

lista_atributo ::= lista_atributo COMA atributo |
              lista_atributo COMA error |
              atributo |
```

```
            error;

atributo ::= WIDTH DOS_PUNTOS operacion_matematica |
        HEIGHT DOS_PUNTOS operacion_matematica |
        POINT_X DOS_PUNTOS operacion_matematica |
        POINT_Y DOS_PUNTOS operacion_matematica |
        ORIENTATION DOS_PUNTOS tipo_orientacion |
        ELEMENTS DOS_PUNTOS LLAVE_ABIERTO lista_elemento LLAVE_CERRADO |
        STYLES CORCHETE_ABIERTO lista_estilo CORCHETE_CERRADO |
        CONTENT DOS_PUNTOS tipo_valor |
        LABEL DOS_PUNTOS tipo_valor |
        OPTIONS DOS_PUNTOS LLAVE_ABIERTO lista_opcion LLAVE_CERRADO |
        OPTIONS DOS_PUNTOS pokemon_func |
        CORRECT DOS_PUNTOS asignacion_correcto;

tipo_orientacion ::= VERTICAL |
            HORIZONTAL;

lista_elemento ::= lista_elemento COMA elemento |
            lista_elemento |
            elemento |
            error;

elemento ::= seccion |
        pregunta |
        tabla |
        texto;

lista_estilo ::= lista_estilo COMA estilo |
          lista_estilo COMA error |
          estilo |
          error;

estilo ::= COLOR DOS_PUNTOS color |
        BACKGROUND_COLOR DOS_PUNTOS color |
        FONT_FAMILY DOS_PUNTOS tipo_fuente |
        TEXT_SIZE DOS_PUNTOS operacion_matematica |
        BORDER DOS_PUNTOS PARENTESIS_ABIERTO operacion_matematica COMA
```

```
tipo_borde COMA color PARENTESIS_CERRADO;

tipo_fuente ::= MONO |
        SANS_SERIF |
        CURSIVE;

tipo_borde ::= LINE |
        DOTTED |
        DOUBLE;


lista_opcion ::= lista_opcion COMA tipo_valor |
        lista_opcion COMA error |
        tipo_valor |
        error;

asignacion_correcto ::= operacion_matematica |
        LLAVE_ABIERTO lista_correcto LLAVE_CERRADO;

lista_correcto ::= lista_correcto COMA operacion_matematica |
        lista_correcto COMA error |
        operacion_matematica |
        error;

pokemon_func ::= WHO_IS_THAT_POKEMON PARENTESIS_ABIERTO
NUMBER_POKEMON COMA operacion_matematica COMA operacion_matematica
PARENTESIS_CERRADO;

seccion ::= SECTION CORCHETE_ABIERTO lista_atributo CORCHETE_CERRADO;

tabla ::= TABLE CORCHETE_ABIERTO lista_atributo CORCHETE_CERRADO;

texto ::= TEXT CORCHETE_ABIERTO lista_atributo CORCHETE_CERRADO;

pregunta ::= OPEN_QUESTION CORCHETE_ABIERTO lista_atributo
CORCHETE_CERRADO |
        DROP_QUESTION CORCHETE_ABIERTO lista_atributo
CORCHETE_CERRADO |
```

```
        SELECT_QUESTION CORCHETE_ABIERTO lista_atributo
    CORCHETE_CERRADO |
        MULTIPLE_QUESTION CORCHETE_ABIERTO lista_atributo
    CORCHETE_CERRADO;

    if ::= IF PARENTESIS_ABIERTO operacion_lra PARENTESIS_CERRADO
LLAVE_ABIERTO contenido LLAVE_CERRADO fin_if;

    fin_if ::= ELSE continuar_else |
        /* empty */;

    continuar_else ::= if |
        LLAVE_ABIERTO contenido LLAVE_CERRADO;

    while ::= WHILE PARENTESIS_ABIERTO operacion_lra PARENTESIS_CERRADO
LLAVE_ABIERTO contenido LLAVE_CERRADO;

    do ::= DO LLAVE_ABIERTO contenido LLAVE_CERRADO WHILE
PARENTESIS_ABIERTO operacion_lra PARENTESIS_CERRADO;

    for ::= FOR PARENTESIS_ABIERTO condicion_for PARENTESIS_CERRADO
LLAVE_ABIERTO contenido LLAVE_CERRADO;

    condicion_for ::= variable PUNTO_COMA operacion_lra PUNTO_COMA variable |
        ID IN operacion_matematica PUNTO PUNTO operacion_matematica;

    operacion_lra ::= operacion_lra OR operacion_and |
        operacion_and;

    operacion_and ::= operacion_and AND operacion_not |
        operacion_not;

    operacion_not ::= NOT operacion_lra |
        operacion_relacional;

    operacion_relacional ::= operacion_matematica operador_relacional
operacion_matematica |
        operacion_matematica;
```

```
operacion_matematica ::= operacion_matematica operador_sr tn |
      tn;

tn ::= tn operador_md g |
      g;

g ::= g operador_pm fn |
      fn;

fn ::= PARENTESIS_ABIERTO operacion_matematica PARENTESIS_CERRADO
|
      NUMERO_DECIMAL |
      NUMERO_ENTERO |
      ID |
      TEXTO |
      COMODIN |
      RESTA fn;

operador_sr ::= SUMA |
      RESTA;

operador_md ::= MULTIPLICACION |
      DIVISION;

operador_pm ::= POTENCIA |
      MODULO;

operador_relacional ::= IGUALDAD |
      DIFERENTE |
      MAYOR |
      MENOR |
      MAYOR_IGUAL |
      MENOR_IGUAL;
```